Informe del Entregable

Concurrencia + EF Core + Reportes

Proyecto: FixTrack — Sistema de Gestión de Reparaciones de Dispositivos Electrónicos

Asignatura: Programación de Aplicaciones de Escritorio | Fecha: 24 de octubre de 2026

Repositorio: github.com/char-lie17/FixTrack | Rama: Concurrencia-Y-EF-Core

1. Objetivo del entregable

Demostrar en FixTrack: (1) **concurrencia y asincronía** en WinForms con tareas en segundo plano, GUI responsiva, manejo de excepciones y cancelación, integradas en funcionalidades reales; (2) **EF Core (Code First)** con `DbContext`, entidades, relaciones, consultas LINQ y una migración; (3) **reportes dinámicos** con datos reales; y (4) **conservación** de las funcionalidades previas y de la arquitectura en capas.

2. Concurrencia y asincronía (WinForms)

2.1 Capa de datos asíncrona

Los **7 DAL** (`ClienteDAL`, `DispositivoDAL`, `OrdenServicioDAL`, `PagoDAL`, `TecnicoDAL`, `UsuarioDAL` y `ReportesDAL`) migraron de ADO.NET sincrónico a EF Core con métodos `async` con firma `*Async(CancellationToken ct = default)`: ninguna operación de base de datos bloquea el hilo de UI.

2.2 Formularios responsivos (19 formularios convertidos)

- **Listados (7):** carga asíncrona con `CancellationTokenSource`; el token se cancela en `FormClosed` para evitar cargas tardías al cerrar el formulario.
- **Guardados:** botón deshabilitado durante la operación, `try/finally` para reactivarlo y mensajes de error sin tumbar la aplicación.
- **Cargas perezosas:** combos de clientes/dispositivos/técnicos con `async` y detalles abiertos en el evento Shown.
- **Dashboard:** métricas en segundo plano notificadas con `IProgress<string>` y etiqueta de estado «Actualizando...».
- **FrmOrdenDetalle:** guardado con detección de **conflicto optimista** (fila afectada = 0).

2.3 Manejo de excepciones y cancelación

`UIHelper.EjecutarSeguroAsync` (y su versión genérica `where T : class`) ejecuta la operación, captura excepciones, muestra el mensaje amigable y distingue `OperationCanceledException` como cancelación limpia y no como error real. El estado del botón se restaura siempre en `finally`; ninguna excepción queda sin capturar en el camino de la UI.

2.4 Integración en funcionalidades reales

La concurrencia no está en código de prueba: se aplicó al alta y edición de clientes, dispositivos, órdenes, pagos, técnicos y usuarios, a la búsqueda y filtrado de los 7 listados, al dashboard y a los reportes.

3. EF Core (Code First)

Elemento	Detalle
Paquete	Entity Framework Core **10.0.13** (proveedor SQL Server)
DbContext	`FixTrackDbContext`: 7 `DbSet`, PK/FK, CHECKs, DEFAULTs e índices (único filtrado `UQ_Usuarios_TecnicoID` e `IX_HistorialOrdenes_OrdenID_FechaCambio DESC`)
Relaciones	Cliente → Dispositivos → OrdenesServicio → Pagos/Historial; Técnico → OrdenesServicio; Técnico ↔ Usuario (1:0..1); borrados *Restrict*
Migración	`APP/Migrations/20261023004419_Inicial.cs` (+ Designer y Snapshot): generada y **aplicada** a la BD
Inicialización	`FixTrackDb.Inicializar()` en `Program.Main` y en `TestRunner`
Consultas LINQ	Búsquedas, filtros, joins y agregaciones en los 7 DAL y en `ReportesDAL`

Se conservaron `Conexion.cs` y `Seguridad.cs` (ADO.NET) para la infraestructura puntual, cumpliendo el requisito de complementar sin romper lo previo.

4. Reportes dinámicos con datos reales

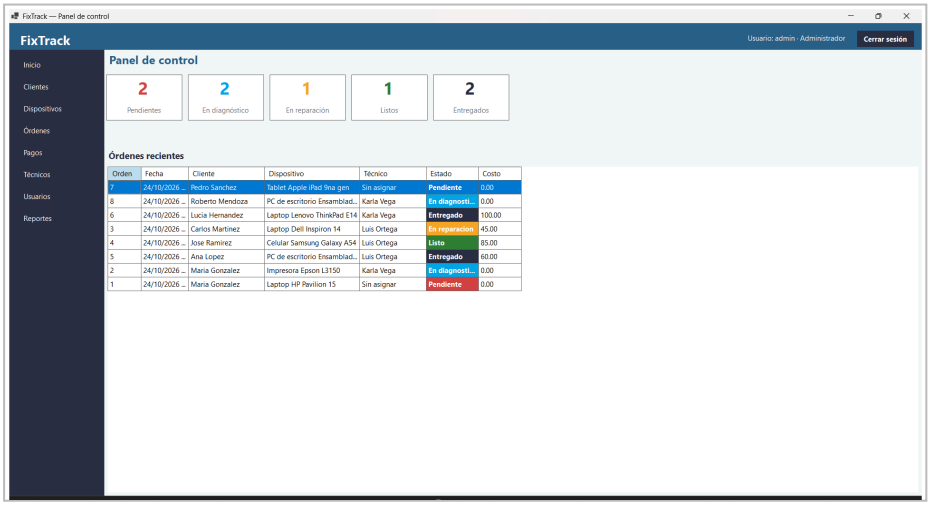
El módulo **Reportes** quedó conectado a `ReportesDAL` (LINQ + `sp_ReporteOrdenesPorEstado`), con 4 reportes, filtros de fecha y exportación a CSV. Se verificó con los datos reales de la BD: el reporte «Ordenes por estado» devuelve la distribución real de las 8 órdenes del sistema (captura 3).

5. Evidencias de funcionamiento

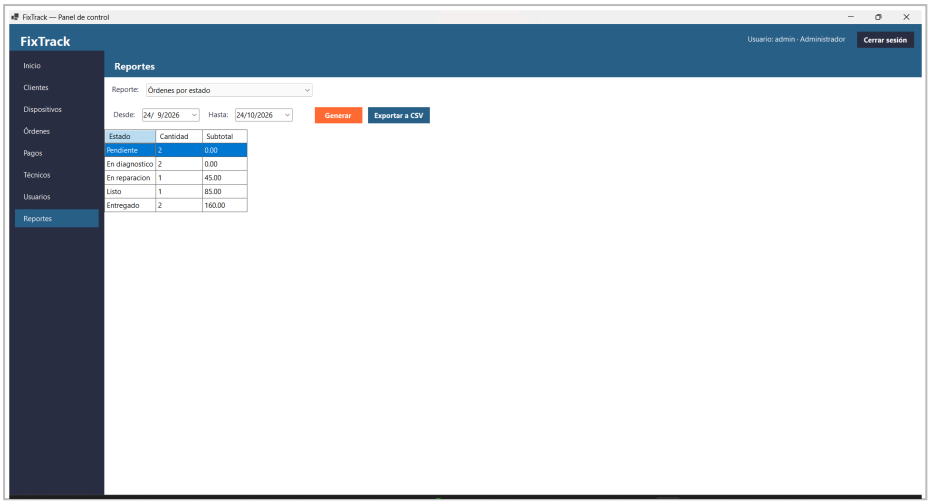
Capturas tomadas sobre la aplicación ejecutándose contra la BD real `FixTrack` con datos de prueba (archivos en la carpeta `Evidencias/`):



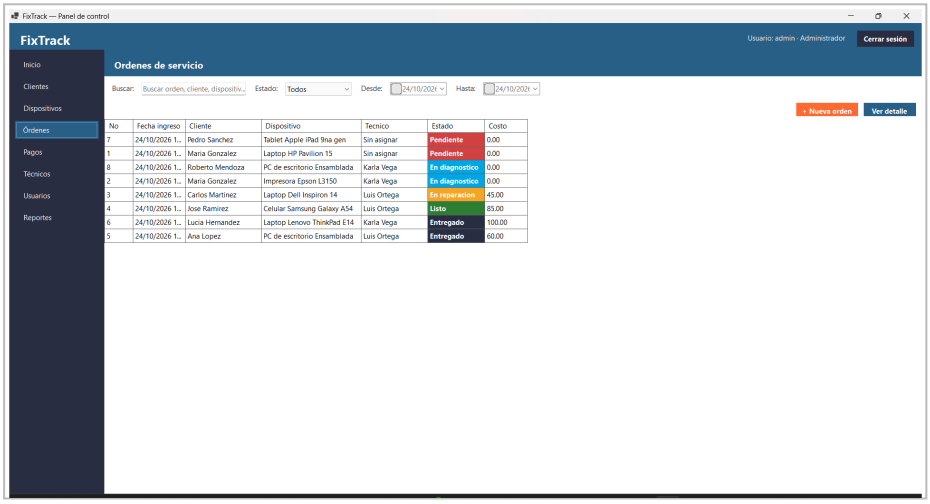
Captura 1. Login de la aplicación (autenticación asíncrona)



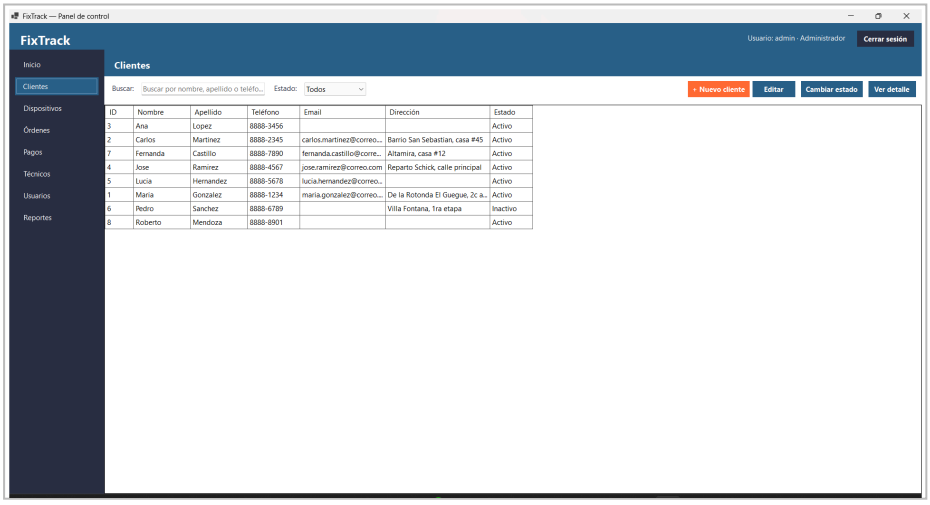
Captura 2. Dashboard con métricas reales cargadas en segundo plano



Captura 3. Reporte dinámico «Ordenes por estado» con datos reales



Captura 4. Listado de órdenes con estados y filtros (carga asíncrona)



Captura 5. Listado de clientes con búsqueda (carga asíncrona)

La carpeta `Evidencias/` incluye además: `06_ejecucion_TestRunner.txt` (salida completa de las pruebas: **44 OK, 0 FALLO(S)**), `07_migraciones_EFCore.txt` (listado de migraciones de `dotnet ef`) y `08_estado_BD_sqlcmd.txt` (migración aplicada en `__EFMigrationsHistory` y conteos reales de las 7 tablas).

Compilación: `dotnet build FixTrack.sln` → 0 errores. **Pruebas de extremo a extremo:** APP/`TestRunner` → RESULTADO: 44 OK, 0 FALLO(S).

6. Base de datos (script e instrucciones)

Dos caminos documentados en `BD/INSTRUCCIONES_RECONSTRUCCION.md`:

- **Opción A (recomendada, Code First):** `dotnet ef database update` + `sqlcmd -i BD\Seed_Datos_Prueba.sql`
- **Opción B (script completo):** `sqlcmd -i BD\FixTrack_BD.sql` — *advertencia: contiene DROP DATABASE y recrea la BD completa.*

Credenciales de prueba: `admin/admin123`, `recepcion1/recepcion123`, `luis.ortega | karla.vega | diego.salinas / tecnico123`.

7. Participación de los integrantes

#	Integrante	Participación (funcionalidades / evidencias)
1	Henry David Lopez Sanchez	**Concurrencia y asincronía en WinForms:** conversión de los 19 formularios a `async/await`, `CancellationTokenSource` en los 7 listados (cancelación en `FormClosed`), guardados con botón deshabilitado y `finally`, y `UIHelper.EjecutarSeguroAsync` con distinción de `OperationCanceledException`.
2	Juan Pablo Sirias Espinoza	**EF Core (Code First):** `FixTrackDbContext` (7 `DbSet`, relaciones, CHECKs, DEFAULTs e índices), fábrica, generación y aplicación de la migración `20261023004419_Inicial`, `FixTrackDb.Inicializar()` y los scripts de seed e instrucciones de BD.
3	Cris Angel Lumbi Rodriguez	**Capa de datos asíncrona:** migración de los 7 DAL a `*Async(CancellationToken ct = default)` con consultas LINQ; carga en segundo plano del Dashboard con `IProgress<string>` y detección de conflicto optimista en `FrmOrdenDetalle`.
4	Gabriel Eduardo Segovia Fonseca	**Reportes dinámicos:** `ReportesDAL` (LINQ + `sp_ReporteOrdenesPorEstado`), 4 reportes con filtros de fecha y exportación a CSV; verificación con datos reales y capturas de las evidencias.
5	Carlos Renato Castillo Lopez	**Coordinación, pruebas y documentación:** suite `TestRunner` (44 pruebas E2E: 44 OK / 0 fallos), revisión de la arquitectura en capas y del cumplimiento de la consigna, contexto del proyecto e informe del entregable.

8. Desviaciones y notas

- La rama se creó como `Concurrencia-Y-EF-Core` (sin espacios) porque Git no admite espacios en nombres de ref.
- `master` no fue modificado en ningún momento; no se hicieron merges ni pull requests.
- No se incluyeron credenciales ni secrets (autenticación integrada local de Windows).
- Las capturas se generaron por automatización de la UI, sin intervención manual, sobre una sesión real de la aplicación.